UNIVERSIDAD TECNOLÓGICA DE PANAMÁ

FACULTAD DE INGENIERÍA DE SISTEMAS COMPUTACIONALES

DEPARTAMENTO DE SISTEMAS COMPUTACIONALES

Desarrollo y Gestión de Software

Desarrollo de software VIII

Laboratorio 2

Profesor

José Chirú

Estudiantes

Roniel Quintero 8-1037-904

Joseph Córdoba 8-1025-2381

Grupo

1GS131

Fecha

17 de octubre de 2026

Problema N°1

Código:

Problema1 > modelos > estudiantes.py > ...

```python

class Estudiantes:
    def __init__(self, cedula, nombre, parcial1, parcial2, proyecto):
        self.cedula = cedula
        self.nombre = nombre
        self.parcial1 = parcial1
        self.parcial2 = parcial2
        self.proyecto = proyecto

    def calcularNotaFinal(self):
        notaFinal = (
            self.parcial1 * 0.30
            + self.parcial2 * 0.30
            + self.proyecto * 0.40
        )
        return round(notaFinal, 2)

    def obtenerCondicion(self):
        if self.calcularNotaFinal() >= 71:
            return "APROBADO"
        else:
            return "REPROBADO"

```

Problema1 > utilidades > controlEstudiante.py > ...

```python
import csv
import json
from modelos.estudiantes import Estudiantes


class ControlEstudiante:
    def __init__(self, archivoEntrada, archivoJSON, archivoResumen):
        self.archivoEntrada = archivoEntrada
        self.archivoJSON = archivoJSON
        self.archivoResumen = archivoResumen
        self.estudiantes = []
        self.errores = []

    def leerArchivo(self):
        self.estudiantes = []
        cedulas = set()

        with open(self.archivoEntrada, "r", newline="", encoding="utf-8") as archivo:
            lector = csv.DictReader(archivo)

            for numero, fila in enumerate(lector, start=2):
                try:
                    cedula = fila["cedula"].strip()
                    nombre = fila["nombre"].strip()

                    if cedula == "" or nombre == "":
                        raise ValueError("La cédula o el nombre están vacíos.")

                    if cedula in cedulas:
                        raise ValueError("Cédula duplicada.")

                    parcial1 = float(fila["parcial1"])
                    parcial2 = float(fila["parcial2"])
                    proyecto = float(fila["proyecto"])

```

```
                    if not (0 <= parcial1 <= 100):
                        raise ValueError("Parcial 1 debe estar entre 0 y 100.")

                    if not (0 <= parcial2 <= 100):
                        raise ValueError("Parcial 2 debe estar entre 0 y 100.")

                    if not (0 <= proyecto <= 100):
                        raise ValueError("El proyecto debe estar entre 0 y 100.")

                    estudiante = Estudiantes(
                        cedula, nombre, parcial1, parcial2, proyecto
                    )

                    self.estudiantes.append(estudiante)
                    cedulas.add(cedula)

                except (ValueError, KeyError) as error:
                    self.errores.append(
                        f"Fila {numero}: {error}"
                    )

        if len(self.estudiantes) < 20:
            self.errores.append(
                "Advertencia: hay menos de 20 estudiantes válidos."
            )

    def calcularEstadisticas(self):
        notas = []

        for estudiante in self.estudiantes:
            notas.append(estudiante.calcularNotaFinal())

        if len(notas) == 0:
            raise ValueError("No hay estudiantes válidos para calcular.")

        promedio = sum(notas) / len(notas)
        notaMayor = max(notas)
        notaMenor = min(notas)

        aprobados = 0
        reprobados = 0
```

```
        for estudiante in self.estudiantes:
            if estudiante.obtenerCondicion() == "APROBADO":
                aprobados += 1
            else:
                reprobados += 1

        return {
            "total_estudiantes": len(self.estudiantes),
            "promedio_general": round(promedio, 2),
            "nota_mayor": notaMayor,
            "nota_menor": notaMenor,
            "aprobados": aprobados,
            "reprobados": reprobados
        }

    def generarJSON(self):
        datos = []

        for estudiante in self.estudiantes:
            registro = {
                "cedula": estudiante.cedula,
                "nombre": estudiante.nombre,
                "parcial1": estudiante.parcial1,
                "parcial2": estudiante.parcial2,
                "proyecto": estudiante.proyecto,
                "nota_final": estudiante.calcularNotaFinal(),
                "condicion": estudiante.obtenerCondicion()
            }

            datos.append(registro)

        with open(self.archivoJSON, "w", encoding="utf-8") as archivo:
            json.dump(datos, archivo, indent=4, ensure_ascii=False)
```

```python
    def generarResumen(self):
        estadisticas = self.calcularEstadisticas()

        with open(self.archivoResumen, "w", encoding="utf-8") as archivo:
            archivo.write("RESUMEN GENERAL DE CALIFICACIONES\n")
            archivo.write("----------------------------------\n")
            archivo.write(
                f"Total de estudiantes: {estadisticas['total_estudiantes']}\n"
            )
            archivo.write(
                f"Promedio general: {estadisticas['promedio_general']}\n"
            )
            archivo.write(f"Nota mayor: {estadisticas['nota_mayor']}\n")
            archivo.write(f"Nota menor: {estadisticas['nota_menor']}\n")
            archivo.write(f"Aprobados: {estadisticas['aprobados']}\n")
            archivo.write(f"Reprobados: {estadisticas['reprobados']}\n")

            if self.errores:
                archivo.write("\nErrores y advertencias:\n")

                for error in self.errores:
                    archivo.write(f"- {error}\n")

    def mostrarResultados(self):
        for estudiante in self.estudiantes:
            print(
                estudiante.cedula,
                estudiante.nombre,
                estudiante.calcularNotaFinal(),
                estudiante.obtenerCondicion()
            )

        print("\nEstadísticas:")
        estadisticas = self.calcularEstadisticas()

        for clave, valor in estadisticas.items():
            print(f"{clave}: {valor}")

        if self.errores:
            print("\nErrores y advertencias:")

            for error in self.errores:
                print(error)

```

Problema1 > main.py > ...

```python
from utilidades.controlEstudiante import ControlEstudiante

archivoEntrada = "estudiantes.csv"
archivoJSON = "estudiantes.json"
archivoResumen = "resumen.txt"

control = ControlEstudiante(
    archivoEntrada,
    archivoJSON,
    archivoResumen
)

try:
    control.
    (method) def mostrarResultados() -> None
    control.mostrarResultados()
    control.generarJSON()
    control.generarResumen()

    print("\nArchivos generados exitosamente.")

except Exception as e:
    print(f"Error en la ejecución => {e}")

```

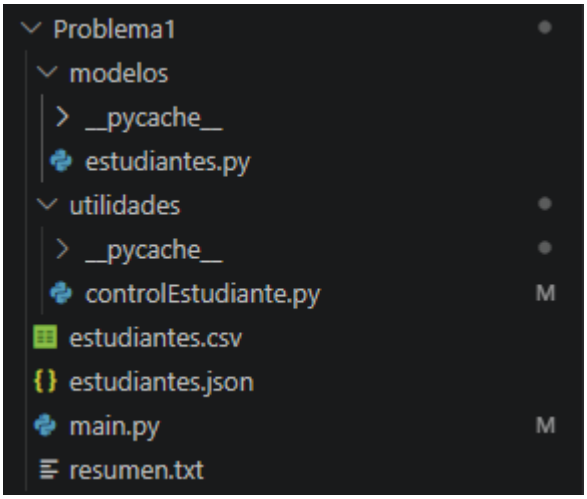
Salidas:

```
PS C:\Users\Sale\OneDrive - Universidad Tecnológica de Panamá\UNIVERSIDAD\III AÑO\II SEMESTRE\DSVIII\LAB2\Problema1> python main.py
 8-123-456 Marcos 90.5 APROBADO
 8-234-567 Ana 72.5 APROBADO
 8-345-678 Luis 91.7 APROBADO
 8-456-789 Sofía 69.6 REPROBADO
 8-567-890 Carlos 81.3 APROBADO
 8-678-901 Maria 90.8 APROBADO
 8-789-012 Pedro 64.9 REPROBADO
 8-890-123 Lucía 84.7 APROBADO
 8-901-234 Diego 70.8 REPROBADO
 8-012-345 Valeria 98.0 APROBADO
 4-123-456 Andres 69.4 REPROBADO
 4-234-567 Camila 87.9 APROBADO
 4-345-678 Javier 77.0 APROBADO
 4-456-789 Isabella 71.6 APROBADO
 4-567-890 Miguel 79.8 APROBADO
 4-678-901 Daniela 92.2 APROBADO
 4-789-012 Gabriel 57.5 REPROBADO
 4-890-123 Paula 82.4 APROBADO
 4-901-234 Ricardo 69.0 REPROBADO
 4-012-345 Elena 86.5 APROBADO

Estadísticas:
total_estudiantes: 20
promedio_general: 79.41
nota_mayor: 98.0
nota_menor: 57.5
aprobados: 14
reprobados: 6

Archivos generados exitosamente.
```

Archivos generados:



main.py M | resumen.txt

Problema1 > resumen.txt

```
1  RESUMEN GENERAL DE CALIFICACIONES
2  ----------------------------------
3  Total de estudiantes: 20
4  Promedio general: 79.41
5  Nota mayor: 98.0
6  Nota menor: 57.5
7  Aprobados: 14
8  Reprobados: 6
9
```

Problema N°2

Código:

main.py Problema1 M | producto.py M | controlInventario.py M | main.py Problema2 M

Problema2 > main.py > ...

```python
from utilidades.controlInventario import ControlInventario

try:
    control = ControlInventario("inventario.json")
    control.cargarInventario()

    print("Inventario cargado correctamente.")
    control.menu()

except FileNotFoundError:
    print("Error: no se encontró el archivo inventario.json.")

except (ValueError, KeyError) as e:
    print(f"Error en los datos del inventario: {e}")

except Exception as e:
    print(f"Error en la ejecución => {e}")

```

main.py M | producto.py M

Problema2 > modelos > producto.py > Producto

```python
class Producto:
    def __init__(self, codigo, producto, cantidad, precio):
        self.codigo = codigo
        self.producto = producto
        self.cantidad = cantidad
        self.precio = precio

    def calcularValor(self):
        return self.cantidad * self.precio

```

Problema2 > utilidades > controlInventario.py > ...

```python
import json
import csv
from modelos.producto import Producto


class ControlInventario:
    def __init__(self, archivoEntrada):
        self.archivoEntrada = archivoEntrada
        self.productos = []
        self.movimientos = []

    def cargarInventario(self):
        with open(self.archivoEntrada, "r", encoding="utf-8") as archivo:
            datos = json.load(archivo)

        codigos = []

        for dato in datos:
            codigo = dato["codigo"]

            if codigo in codigos:
                raise ValueError("Hay códigos duplicados en el JSON.")

            cantidad = int(dato["cantidad"])
            precio = float(dato["precio"])

            if cantidad < 0 or precio < 0:
                raise ValueError("La cantidad y el precio no pueden ser negativos.")

            producto = Producto(
                codigo,
                dato["producto"],
                cantidad,
                precio
            )

            self.productos.append(producto)
            codigos.append(codigo)

    def listarProductos(self):
        print("\n--- INVENTARIO ---")
        for producto in self.productos:
            print(
                producto.codigo,
                producto.producto,
                "Cantidad:", producto.cantidad,
                "Precio:", producto.precio
            )

    def buscarProducto(self, codigo):
        for producto in self.productos:
            if producto.codigo == codigo:
                return producto

        return None

    def registrarMovimiento(self, codigo, tipo, cantidad):
        producto = self.buscarProducto(codigo)

        if producto is None:
            print("Producto no encontrado.")
            return

        if cantidad <= 0:
            print("La cantidad del movimiento debe ser positiva.")
            return

        if tipo == "entrada":
            producto.cantidad += cantidad

        elif tipo == "salida":
            if cantidad > producto.cantidad:
                print("No hay suficientes existencias.")
                return

            producto.cantidad -= cantidad

        else:
            print("Tipo de movimiento inválido.")
            return
```

```
        movimiento = {
            "codigo": producto.codigo,
            "tipo": tipo,
            "cantidad": cantidad,
            "existencia_final": producto.cantidad
        }

        self.movimientos.append(movimiento)
        print("Movimiento registrado correctamente.")

    def calcularValorInventario(self):
        total = 0

        for producto in self.productos:
            total += producto.calcularValor()

        return total

    def mostrarExistenciasBajas(self):
        print("\n--- PRODUCTOS CON MENOS DE 15 UNIDADES ---")
        encontrados = 0

        for producto in self.productos:
            if producto.cantidad < 15:
                print(producto.codigo, producto.producto,
                      "Cantidad:", producto.cantidad)
                encontrados += 1

        if encontrados == 0:
            print("No hay productos con existencias bajas.")

    def guardarInventarioJSON(self):
        datos = []

        for producto in self.productos:
            registro = {
                "codigo": producto.codigo,
                "producto": producto.producto,
                "cantidad": producto.cantidad,
                "precio": producto.precio
            }
            datos.append(registro)
```

```
126         with open("inventario_actualizado.json", "w", encoding="utf-8") as archivo:
127             json.dump(datos, archivo, indent=4, ensure_ascii=False)
128 
129     def guardarInventarioCSV(self):
130         with open("inventario.csv", "w", newline="", encoding="utf-8") as archivo:
131             escritor = csv.writer(archivo)
132             escritor.writerow(["codigo", "producto", "cantidad", "precio"])
133 
134             for producto in self.productos:
135                 escritor.writerow([
136                     producto.codigo,
137                     producto.producto,
138                     producto.cantidad,
139                     producto.precio
140                 ])
141 
142     def guardarMovimientosCSV(self):
143         with open("movimientos.csv", "w", newline="", encoding="utf-8") as archivo:
144             escritor = csv.writer(archivo)
145             escritor.writerow([
146                 "codigo", "tipo", "cantidad", "existencia_final"
147             ])
148 
149             for movimiento in self.movimientos:
150                 escritor.writerow([
151                     movimiento["codigo"],
152                     movimiento["tipo"],
153                     movimiento["cantidad"],
154                     movimiento["existencia_final"]
155                 ])
156 
157     def menu(self):
158         opcion = ""
159 
160         while opcion != "6":
161             print("\n===== MENU DE INVENTARIO =====")
162             print("1. Listar productos")
163             print("2. Buscar producto por código")
164             print("3. Registrar entrada")
165             print("4. Registrar salida")
166             print("5. Mostrar valor y existencias bajas")
167             print("6. Guardar archivos y salir")
```

```python
            opcion = input("Seleccione una opción: ")

            if opcion == "1":
                self.listarProductos()

            elif opcion == "2":
                codigo = input("Ingrese el código: ")
                producto = self.buscarProducto(codigo)

                if producto is None:
                    print("Producto no encontrado.")
                else:
                    print(
                        producto.codigo,
                        producto.producto,
                        "Cantidad:", producto.cantidad,
                        "Precio:", producto.precio
                    )

            elif opcion == "3" or opcion == "4":
                codigo = input("Ingrese el código del producto: ")

                try:
                    cantidad = int(input("Ingrese la cantidad: "))

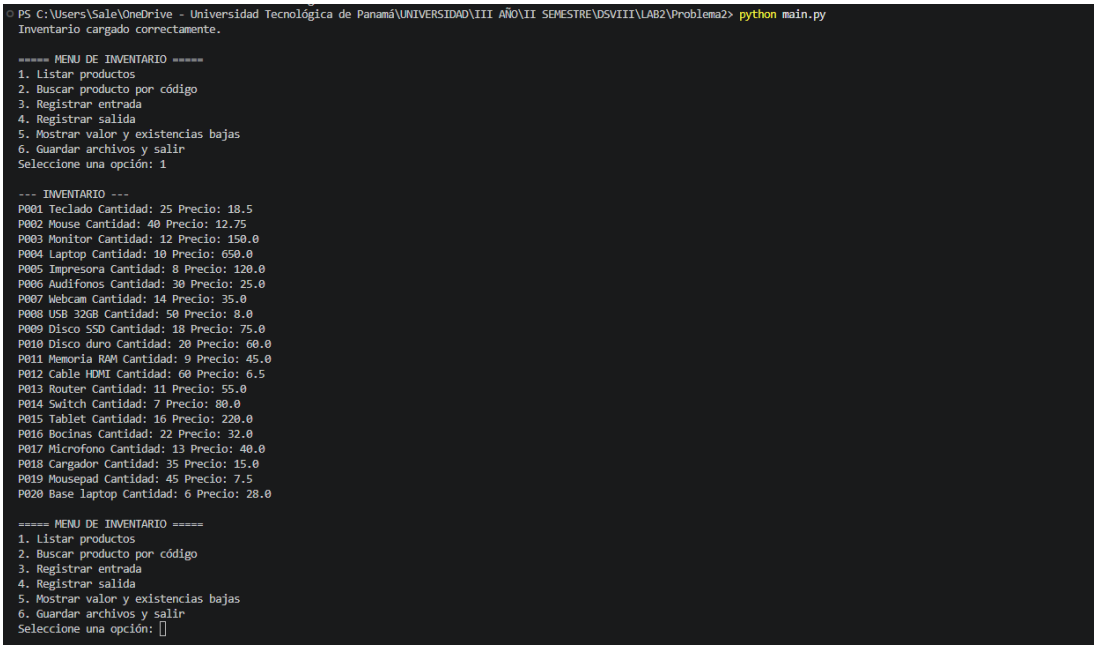
                    if opcion == "3":
                        self.registrarMovimiento(codigo, "entrada", cantidad)
                    else:
                        self.registrarMovimiento(codigo, "salida", cantidad)

                except ValueError:
                    print("Debe ingresar una cantidad entera válida.")

            elif opcion == "5":
                print(
                    "Valor total del inventario:",
                    round(self.calcularValorInventario(), 2)
                )
                self.mostrarExistenciasBajas()

            elif opcion == "6":
                self.guardarInventarioJSON()
                self.guardarInventarioCSV()
                self.guardarMovimientosCSV()
                print("Archivos guardados correctamente.")
```

Salidas en terminal:



```
PS C:\Users\Sale\OneDrive - Universidad Tecnológica de Panamá\UNIVERSIDAD\III AÑO\II SEMESTRE\DSVIII\LAB2\Problema2> python main.py
Inventario cargado correctamente.

===== MENU DE INVENTARIO =====
1. Listar productos
2. Buscar producto por código
3. Registrar entrada
4. Registrar salida
5. Mostrar valor y existencias bajas
6. Guardar archivos y salir
Seleccione una opción: 1

--- INVENTARIO ---
P001 Teclado Cantidad: 25 Precio: 18.5
P002 Mouse Cantidad: 40 Precio: 12.75
P003 Monitor Cantidad: 12 Precio: 150.0
P004 Laptop Cantidad: 10 Precio: 650.0
P005 Impresora Cantidad: 8 Precio: 120.0
P006 Audifonos Cantidad: 30 Precio: 25.0
P007 Webcam Cantidad: 14 Precio: 35.0
P008 USB 32GB Cantidad: 50 Precio: 8.0
P009 Disco SSD Cantidad: 18 Precio: 75.0
P010 Disco duro Cantidad: 20 Precio: 60.0
P011 Memoria RAM Cantidad: 9 Precio: 45.0
P012 Cable HDMI Cantidad: 60 Precio: 6.5
P013 Router Cantidad: 11 Precio: 55.0
P014 Switch Cantidad: 7 Precio: 80.0
P015 Tablet Cantidad: 16 Precio: 220.0
P016 Bocinas Cantidad: 22 Precio: 32.0
P017 Microfono Cantidad: 13 Precio: 40.0
P018 Cargador Cantidad: 35 Precio: 15.0
P019 Mousepad Cantidad: 45 Precio: 7.5
P020 Base laptop Cantidad: 6 Precio: 28.0

===== MENU DE INVENTARIO =====
1. Listar productos
2. Buscar producto por código
3. Registrar entrada
4. Registrar salida
5. Mostrar valor y existencias bajas
6. Guardar archivos y salir
Seleccione una opción:
```

```
===== MENU DE INVENTARIO =====
1. Listar productos
2. Buscar producto por código
3. Registrar entrada
4. Registrar salida
5. Mostrar valor y existencias bajas
6. Guardar archivos y salir
Seleccione una opción: 2
Ingrese el código: P005
P005 Impresora Cantidad: 8 Precio: 120.0

===== MENU DE INVENTARIO =====
1. Listar productos
2. Buscar producto por código
3. Registrar entrada
4. Registrar salida
5. Mostrar valor y existencias bajas
6. Guardar archivos y salir
Seleccione una opción:
```

```
===== MENU DE INVENTARIO =====
1. Listar productos
2. Buscar producto por código
3. Registrar entrada
4. Registrar salida
5. Mostrar valor y existencias bajas
6. Guardar archivos y salir
Seleccione una opción: 3
Ingrese el código del producto: P005
Ingrese la cantidad: 99
Movimiento registrado correctamente.

===== MENU DE INVENTARIO =====
1. Listar productos
2. Buscar producto por código
3. Registrar entrada
4. Registrar salida
5. Mostrar valor y existencias bajas
6. Guardar archivos y salir
Seleccione una opción:
```

```
===== MENU DE INVENTARIO =====
1. Listar productos
2. Buscar producto por código
3. Registrar entrada
4. Registrar salida
5. Mostrar valor y existencias bajas
6. Guardar archivos y salir
Seleccione una opción: 4
Ingrese el código del producto: P011
Ingrese la cantidad: 85
No hay suficientes existencias.

===== MENU DE INVENTARIO =====
1. Listar productos
2. Buscar producto por código
3. Registrar entrada
4. Registrar salida
5. Mostrar valor y existencias bajas
6. Guardar archivos y salir
Seleccione una opción:
```

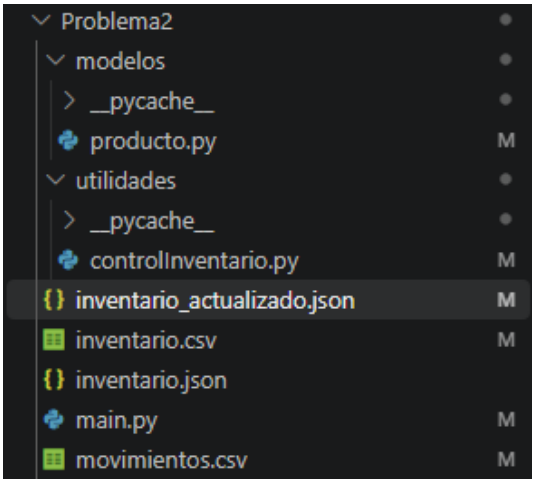
```
 ===== MENU DE INVENTARIO =====
 1. Listar productos
 2. Buscar producto por código
 3. Registrar entrada
 4. Registrar salida
 5. Mostrar valor y existencias bajas
 6. Guardar archivos y salir
 Seleccione una opción: 5
 Valor total del inventario: 34037.0

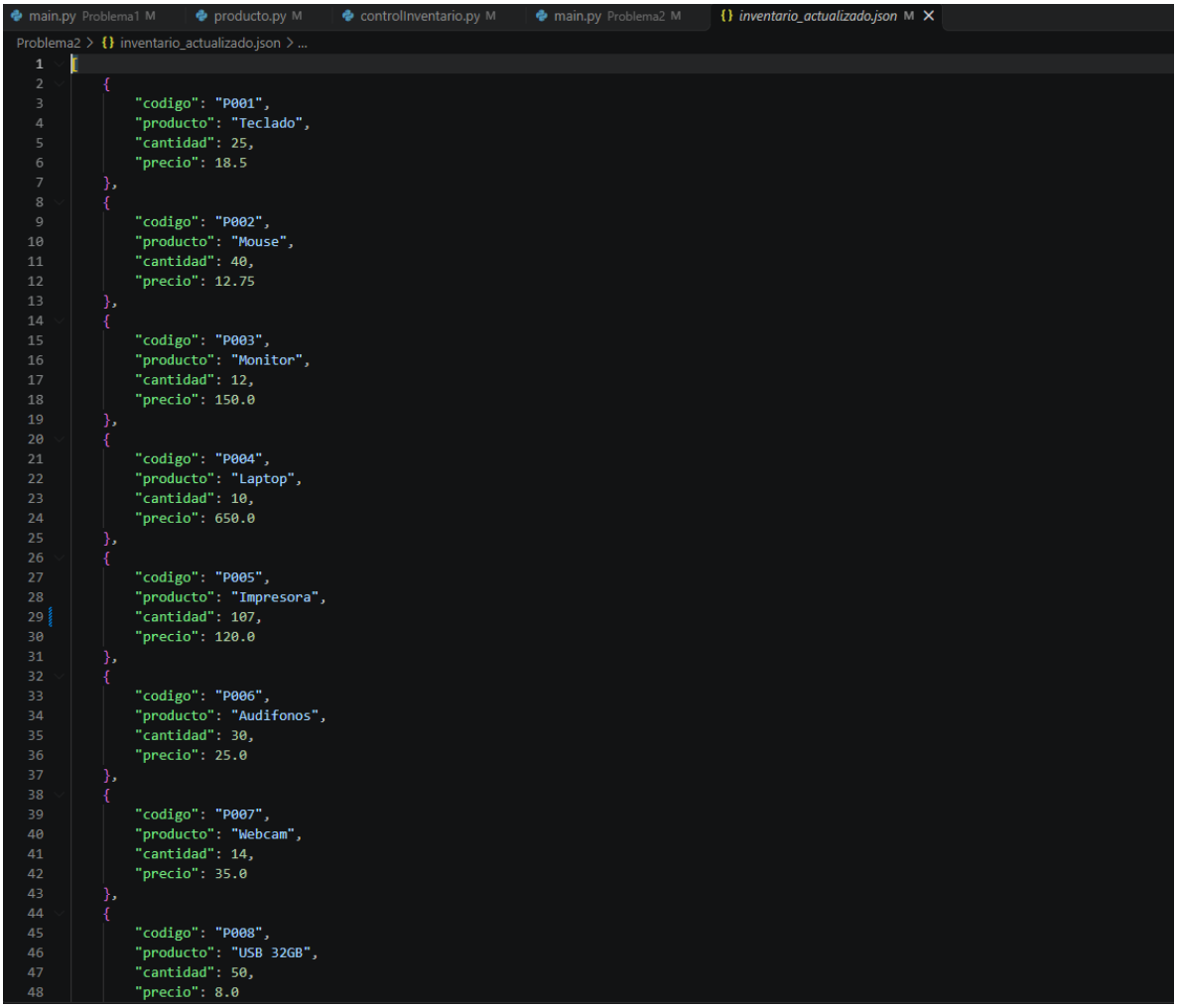
 --- PRODUCTOS CON MENOS DE 15 UNIDADES ---
 P003 Monitor Cantidad: 12
 P004 Laptop Cantidad: 10
 P007 Webcam Cantidad: 14
 P011 Memoria RAM Cantidad: 9
 P013 Router Cantidad: 11
 P014 Switch Cantidad: 7
 P017 Microfono Cantidad: 13
 P020 Base laptop Cantidad: 6

 ===== MENU DE INVENTARIO =====
 1. Listar productos
 2. Buscar producto por código
 3. Registrar entrada
 4. Registrar salida
 5. Mostrar valor y existencias bajas
 6. Guardar archivos y salir
 Seleccione una opción:
```

```
===== MENU DE INVENTARIO =====
1. Listar productos
2. Buscar producto por código
3. Registrar entrada
4. Registrar salida
5. Mostrar valor y existencias bajas
6. Guardar archivos y salir
Seleccione una opción: 6
Archivos guardados correctamente.
PS C:\Users\Sale\OneDrive - Universidad Tecnológica de Panamá\UNIVERSIDAD\III AÑO\II SEMESTRE\DSVIII\LAB2\Problema2>
```

Archivos generados:






```json
[
    {
        "codigo": "P001",
        "producto": "Teclado",
        "cantidad": 25,
        "precio": 18.5
    },
    {
        "codigo": "P002",
        "producto": "Mouse",
        "cantidad": 40,
        "precio": 12.75
    },
    {
        "codigo": "P003",
        "producto": "Monitor",
        "cantidad": 12,
        "precio": 150.0
    },
    {
        "codigo": "P004",
        "producto": "Laptop",
        "cantidad": 10,
        "precio": 650.0
    },
    {
        "codigo": "P005",
        "producto": "Impresora",
        "cantidad": 107,
        "precio": 120.0
    },
    {
        "codigo": "P006",
        "producto": "Audifonos",
        "cantidad": 30,
        "precio": 25.0
    },
    {
        "codigo": "P007",
        "producto": "Webcam",
        "cantidad": 14,
        "precio": 35.0
    },
    {
        "codigo": "P008",
        "producto": "USB 32GB",
        "cantidad": 50,
        "precio": 8.0
```

Problema N°3

Análisis de sensores IoT con Polars

Código:

```python
import polars as pl
import sys

class AnalizadorIoT:
    def __init__(self, ruta_archivo):
        self.ruta = ruta_archivo
        self.df = None
        self.resumen = None
        self.alertas = None

    def cargar_y_validar(self):
        """Lee el archivo CSV y valida los registros."""
        try:
            # Leer archivo de sensores con Polars
            df_crudo = pl.read_csv(self.ruta)

            # Validar identificadores y registros incompletos eliminando nulos
            df_validado = df_crudo.drop_nulls()

            # Validar humedad entre 0 y 100
            self.df = df_validado.filter(
                (pl.col('humedad') >= 0) & (pl.col('humedad') <= 100)
            )
            print("Datos cargados y validados correctamente.")

        except FileNotFoundError:
            print(f"Error: El archivo '{self.ruta}' no existe o no fue encontrado.")
            sys.exit(1)
        except Exception as e:
            print(f"Error inesperado al leer el archivo: {e}")
            sys.exit(1)
```



```python
def procesar_alertas(self):
        """Crea la columna de estado y define los motivos de las alertas."""
        if self.df is None: return

        # Crear columna de estado: ALERTA si temp > 32.0 o humedad > 80%; NORMAL en otro caso
        self.df = self.df.with_columns(
            pl.when((pl.col('temperatura') > 32.0) | (pl.col('humedad') > 80))
            .then(pl.lit('ALERTA'))
            .otherwise(pl.lit('NORMAL'))
            .alias('estado')
        )

        # Incluir el motivo que activó cada alerta
        self.df = self.df.with_columns(
            pl.when((pl.col('temperatura') > 32.0) & (pl.col('humedad') > 80))
            .then(pl.lit('Temperatura > 32.0°C y Humedad > 80%'))
            .when(pl.col('temperatura') > 32.0)
            .then(pl.lit('Temperatura > 32.0°C'))
            .when(pl.col('humedad') > 80)
            .then(pl.lit('Humedad > 80%'))
            .otherwise(pl.lit('N/A'))
            .alias('motivo')
        )

    def generar_estadisticas(self):
        """Calcula agregaciones por sensor y ubicación."""
        if self.df is None: return

        # Agrupar para calcular cantidad, temperatura (promedio, max, min) y humedad promedio
        self.resumen = self.df.group_by(['sensor', 'ubicacion']).agg([
            pl.col('sensor').count().alias('cantidad_mediciones'),
            pl.col('temperatura').mean().alias('temp_promedio'),
            pl.col('temperatura').max().alias('temp_maxima'),
            pl.col('temperatura').min().alias('temp_minima'),
            pl.col('humedad').mean().alias('humedad_promedio')
        ])
```

```python
def exportar_archivos(self):
            """Filtra las alertas y exporta los archivos CSV y JSON requeridos."""
            if self.df is None or self.resumen is None: return

            # Filtrar solo los registros que tienen estado de alerta
            self.alertas = self.df.filter(pl.col('estado') == 'ALERTA')

            try:
                # 1. Exportar alertas.json
                self.alertas.write_json('alertas.json')

                # 2. Exportar resumen_sensores.csv
                self.resumen.write_csv('resumen_sensores.csv')

                # 3. Exportar sensores_procesados.json
                self.df.write_json('sensores_procesados.json')

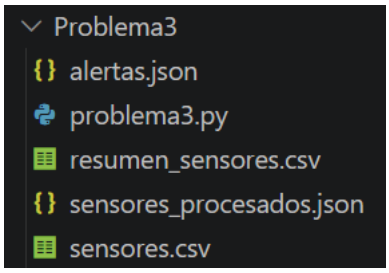
                print("Los archivos alertas.json, resumen_sensores.csv y sensores_procesados.json fueron generados con éxito.")
            except Exception as e:
                print(f"Error al exportar los archivos: {e}")

# Bloque de ejecución principal
if __name__ == '__main__':
    analizador = AnalizadorIoT('sensores.csv')
    analizador.cargar_y_validar()
    analizador.procesar_alertas()
    analizador.generar_estadisticas()
    analizador.exportar_archivos()
```

Salida en terminal:

```
PS C:\Desarrollo8\Laboratorio2_DSVIII\Problema3> python Problema3.py
Datos cargados y validados correctamente.
Los archivos alertas.json, resumen_sensores.csv y sensores_procesados.json fueron generados con éxito.
PS C:\Desarrollo8\Laboratorio2_DSVIII\Problema3>
```

Archivos:



problema3.py | resumen_sensores.csv

Problema3 > resumen_sensores.csv

```
sensor,ubicacion,cantidad_mediciones,temp_promedio,temp_maxima,temp_minima,humedad_promedio
S03,Edificio C,4,32.65,34.0,31.0,82.5
S05,Edificio B,3,28.900000000000002,29.5,28.2,67.66666666666667
S04,Edificio A,3,27.5,28.0,27.0,62.0
S01,Edificio A,5,29.9,32.5,28.5,78.6
S02,Edificio B,5,31.840000000000003,33.5,30.2,80.6
```

problema3.py sensores_procesados.json

Problema3 > sensores_procesados.json > ...

```
1  [{"sensor":"S01","ubicacion":"Edificio A","fecha":"2026-10-01","hora":"08:00","temperatura":28.5,
```

problema3.py alertas.json

Problema3 > alertas.json > ...

```
1  [{"sensor":"S03","ubicacion":"Edificio C","fecha":"2026-10-01","hora":"08:15","temperatura":33.1,
```

Conclusiones

Mediante el desarrollo de este laboratorio se logró poner en práctica el manejo e intercambio de datos mediante archivos CSV y JSON utilizando Python. A través de los tres problemas propuestos, se trabajó en la lectura, validación y procesamiento de información, así como en la generación de archivos de salida para almacenar y compartir los resultados obtenidos.

Además, se aplicó la programación orientada a objetos mediante la creación de clases y métodos, lo que permitió organizar el código de manera estructurada. También se reforzó el uso de estructuras de control, listas y manejo de excepciones para controlar datos inválidos, evitar duplicados y realizar operaciones de forma adecuada.